

Chapter 31 - Coprocessor and Cross-CPU Calls

The six processors of Part IV all share Intuition Engine's memory bus and MMIO map. They can therefore cooperate: a BASIC program running on IE64 can hand a piece of work to the 6502, and a 68K program can ask IE64 to compute a hard floating-point expression. The mechanism is the **coprocessor system**: a set of MMIO registers, a per-CPU worker queue, and three BASIC verbs.

This chapter also covers the **image executor** - the small region at 0xF2320 that the RUN command uses to launch executable images written for any of the six CPUs.

31.1 The image executor

When BASIC executes RUN "filename.ie65", BASIC does not load the file itself. It writes the filename pointer and an "execute" command to the image executor block at 0xF2320-0xF233F. The executor:

1. Looks at the filename extension to determine which CPU should run the file (.ie65 → 6502, .ie80 → Z80, .ie32 → IE32, .ie64 → IE64, .ie68 → M68K, .ie86 → x86).
2. Reads the file through the bootstrap file bridge (Chapter 23 §23.6).
3. Loads the file's payload into RAM at PROG_START = 0x1000.
4. Triggers a reset on the chosen CPU.
5. Switches the active CPU to that one.

When the executed program halts (typically by writing 0xDEAD to TERM_SENTINEL), the executor switches the active CPU back to whatever was running before and BASIC resumes.

Direct-mode RUN "filename" from BASIC and the RUN form with a file extension both route through the executor. The numeric-line forms of RUN (such as RUN 100) do not - they always restart the current BASIC program.

The executor's MMIO registers are not normally accessed by user code. They are documented in the source for advanced use.

31.2 Coprocessor model

A **coprocessor** in Intuition Engine is one of the six CPUs acting as a worker for another CPU. A coprocessor is started, a program runs on it as a service, requests are enqueued through a ring buffer, and replies are written back.

The state of a coprocessor:

- **Stopped** - no worker active for this CPU type.
- **Running** - a worker is loaded; it accepts requests.
- **Faulted** - the worker has stopped responding (a timeout or a service error).

CPU type codes match the image-executor's extension codes:

CPU type	Code	Image extension
IE32	1	.ie32
IE64	2	.ie64
6502	3	.ie65
M68K	4	.ie68

CPU type	Code	Image extension
Z80	5	.ie80
x86	6	.ie86

31.3 BASIC verbs

Three BASIC statements drive the coprocessor system. Two are function-like (COCALL, COSTATUS); the rest are statement-form commands.

31.3.1 COSTART

```
COSTART cpuType, "serviceFile"
```

Loads serviceFile into the target CPU's image area, resets that CPU, and starts it running. The file must be an executable image for the named CPU (COSTART 3, "audio.ie65" starts a 6502 worker from audio.ie65).

COSTART returns immediately; the worker is asynchronous from the caller.

31.3.2 COSTOP

```
COSTOP cpuType
```

Asks the worker for cpuType to shut down cleanly. The worker finishes the request it is currently handling, then exits. Pending tickets in its queue are completed with status COPROC_TICKET_WORKER_DOWN.

31.3.3 COCALL

```
ticket = COCALL(cpuType, op, reqPtr, reqLen, respPtr, respCap)
```

Enqueues a request to the named coprocessor. The request is a block of reqLen bytes starting at reqPtr; the worker's reply will be written to up to respCap bytes at respPtr. The op argument is an application-defined 32-bit code that the worker uses to dispatch.

COCALL returns a non-negative ticket number on success, or a negative value on error. The ticket is used by COSTATUS and COWAIT to track completion.

31.3.4 COSTATUS

```
status = COSTATUS(ticket)
```

Polls a ticket without blocking. Returns:

Value	Constant	Meaning
0	COPROC_TICKET_PENDING	Queued, not yet started
1	COPROC_TICKET_RUNNING	Worker is processing
2	COPROC_TICKET_OK	Completed successfully
3	COPROC_TICKET_ERROR	Worker returned an error

Value	Constant	Meaning
4	COPROC_TICKET_TIMEOUT	Worker exceeded its deadline
5	COPROC_TICKET_WORKER_DOWN	Worker is no longer running

31.3.5 COWAIT

COWAIT ticket [, timeoutMs]

Blocks until the ticket completes or timeoutMs milliseconds elapse. Returns the same status codes as CSTATUS.

Without timeoutMs, COWAIT blocks indefinitely.

31.4 MMIO register block

The coprocessor occupies 0xF2340–0xF238F plus an extension block at 0xF23B0–0xF23BF.

Address	Name	Direction	Purpose
0xF2340	COPROC_CMD	W	Command to execute (see §31.5)
0xF2344	COPROC_CPU_TYPE	W	Selects target CPU type
0xF2348	COPROC_CMD_STATUS	R	0 ok, 1 error
0xF234C	COPROC_CMD_ERROR	R	Error code (see §31.6)
0xF2350	COPROC_TICKET	R/W	Ticket ID (output of ENQUEUE, input to POLL/WAIT)
0xF2354	COPROC_TICKET_STATUS	R	Last-poll ticket status
0xF2358	COPROC_OP	W	Application op code for request
0xF235C	COPROC_REQ_PTR	W	Request buffer pointer
0xF2360	COPROC_REQ_LEN	W	Request buffer length
0xF2364	COPROC_RESP_PTR	W	Response buffer pointer
0xF2368	COPROC_RESP_CAP	W	Response buffer capacity
0xF236C	COPROC_TIMEOUT	W	Timeout in milliseconds (for WAIT)
0xF2370	COPROC_NAME_PTR	W	Pointer to service-file name string
0xF2374	COPROC_WORKER_STATE	R	Bitmask of running workers
0xF2378	COPROC_STATS_OPS	R	Total ops dispatched
0xF237C	COPROC_STATS_BYTES	R	Total bytes processed
0xF2380	COPROC_IRQ_CTRL	R/W	b0 enable IRQ on completion
0xF2384	COPROC_DISPATCH_OVERHEAD	R	Calibrated overhead, nanoseconds
0xF2388	COPROC_COMPLETED_TICKET	R	Last completed ticket ID
0xF23B0	COPROC_RING_DEPTH	R	Selected CPU's ring occupancy
0xF23B4	COPROC_WORKER_UPTIME	R	Worker uptime in seconds
0xF23B8	COPROC_STATS_RESET	W	Write 1 to zero stats

Address	Name	Direction	Purpose
0xF23BC	COPROC_BUSY_PCT	R	Aggregate worker busy % over last second

Reads from COPROC_RING_DEPTH and COPROC_WORKER_UPTIME require writing COPROC_CPU_TYPE first to select which worker to query.

The 6502 and Z80 cannot reach 0xF2340 directly (their 16-bit address space stops at 0xFFFF). For those CPUs a gateway window is mapped at 0xF200–0xF24F and forwarded to the bus.

31.5 Commands

COPROC_CMD accepts the following codes:

Code	Constant	Effect
1	COPROC_CMD_START	Start a worker (COSTART)
2	COPROC_CMD_STOP	Stop a worker (COSTOP)
3	COPROC_CMD_ENQUEUE	Submit a request, returns ticket
4	COPROC_CMD_POLL	Poll a ticket's status
5	COPROC_CMD_WAIT	Block until ticket completes or timeout

Before issuing a command, write the relevant input registers (COPROC_CPU_TYPE, COPROC_NAME_PTR, COPROC_REQ_PTR, etc.). After the write to COPROC_CMD, read COPROC_CMD_STATUS. A non-zero status means failure; read COPROC_CMD_ERROR for the reason.

31.6 Error codes

Code	Constant	Meaning
0	COPROC_ERR_NONE	Success
1	COPROC_ERR_INVALID_CPU	COPROC_CPU_TYPE is not in 1–6
2	COPROC_ERR_NOT_FOUND	Service file does not exist
3	COPROC_ERR_PATH_INVALID	Filename is malformed
4	COPROC_ERR_LOAD_FAILED	Image header is bad or load failed
5	COPROC_ERR_QUEUE_FULL	Worker's ring buffer has no free slots
6	COPROC_ERR_NO_WORKER	No worker is running for this CPU type
7	COPROC_ERR_STALE_TICKET	Ticket has been reaped

31.7 The ring buffer

Each worker has a small ring buffer in shared RAM (16 slots per CPU). The application program does not normally touch the ring directly; COCALL enqueues, COSTATUS/COWAIT read responses. The shared layout is documented in the source for callers that need to bypass the BASIC verbs:

- Ring base for each CPU: $0x790000 + \text{cputype} * 0x300$.

- Per ring: head byte, tail byte, capacity byte, entries (16×32 -byte request descriptors), responses (16×16 -byte response descriptors).

A COCALL writes a request descriptor at the ring head, advances the head, and returns the descriptor's ticket. The worker reads from the tail; when it has produced a response, it writes the response descriptor and advances the response tail.

31.8 The shape of a coprocessor call

The pattern, with `<worker.ie65>` standing for whichever worker image you have written for the 6502:

```
10 REM start 6502 worker (assumes <worker.ie65> exists)
20 COSTART 3, "<worker.ie65>"
30 REM submit a request: op=1, buf at $30000 (in size 4), reply at $30100 (cap 4)
40 POKE &H30000, 42
50 T = COCALL(3, 1, &H30000, 4, &H30100, 4)
60 IF T < 0 THEN PRINT "submit failed": END
70 COWAIT T, 1000
80 IF COSTATUS(T) <> 2 THEN PRINT "compute failed": END
90 PRINT "answer is "; PEEK(&H30100)
100 COSTOP 3
```

A worker image is a standalone 6502 (or other CPU) program that watches its ring, reads request descriptors, processes them, and writes response descriptors back. Writing one is a small project on its own; the BASIC side shown above is the same regardless of which worker is loaded.

31.9 IRQ on completion

Bit 0 of `COPROC_IRQ_CTRL` enables an interrupt when any ticket completes. The interrupt vector depends on the listening CPU (Chapter 30 §30.3). The handler reads `COPROC_COMPLETED_TICKET` to learn which ticket finished.

This is useful when the listening CPU has other work to do and does not want to block in `COWAIT`. The classic pattern is to enqueue several requests, return to other work, and let the completion interrupt wake up the response handler.

31.10 What comes next

Chapter 32 covers the Machine Monitor - the interactive debugger that lets you single-step any CPU, examine registers, set breakpoints and watchpoints, and inspect the bus from a command prompt. Chapter 33 covers IE Script, the scripting surface that drives the monitor from BASIC-like programs.